

제16강

1) Firebase Firestore Database 이용한 댓글 등록

ToDoList16

1. [Firestore Database]-[색인]-[복합]-[색인추가]을 선택한 후 복합 인덱스를 생성한다.

컬렉션ID: 'comments'-'postId(오름차순)'-'date(내림차순)'

2. 높이가 자동으로 증가하는 TEXTAREA 상자 라이브러리를 설치한다.

`npm install react-textarea-autosize`

[src]-[comments16]-[post] ReadPage.jsx

```
import CommentRegister from './comment/CommentRegister'
import CommentList from './comment/CommentList'
...
const ReadPage = () => {
  <Col md={10} lg={9} xl={8}>
    ...
    <CommentRegister postId={ id }/>
    <CommentList postId={ id }/>
  </Col>
}
```

[src]-[comments16]-[post]-[comment] CommentRegister.jsx

```
import React, { useContext, useState } from 'react'
import { Button } from 'react-bootstrap';
import TextareaAutosize from 'react-textarea-autosize'
import { ModalContext } from '../../../context/ModalContext'
import { useNavigate } from 'react-router-dom'
import moment from 'moment'
import { app } from '../../../initFirebase'
import { getFirestore, addDoc, collection } from 'firebase/firestore';

const CommentRegister = ({ postId }) => {
  const email = sessionStorage.getItem('email');
  const { setAlert } = useContext(ModalContext);
  const navi = useNavigate();

  const [content, setContent] = useState('');
  const db = getFirestore(app);

  const onRegister = async() => {
    const date = moment(new Date()).format('YYYY-MM-DD HH:mm:ss');
    await addDoc(collection(db, 'comments'), { postId, content, email, date });
    setAlert({show:true, message:'새로운 댓글이 등록되었습니다.'});
    setContent('');
  }

  return (
    <div className='my-5'>
      {email ?
        <React.Fragment>
          <TextareaAutosize value={content} onChange={(e)=>setContent(e.target.value)}
            placeholder='댓글 내용을 입력하세요.' className='textarea auto' minRows={5}/>
          <Button onClick={ onRegister } disabled={content===''} className='px-5'>등록</Button>
        </React.Fragment>
        :
        <Button onClick={()=>navi('/login')} className='w-100' variant='outline-primary'>댓글쓰기(로그인)</Button>
      }
    </div>
  )
}
```

2) Firebase Firestore Database 이용한 댓글 목록, 삭제 수정

```
[src]-[comments16]-[post]-[comment] CommentList.jsx
```

```
import React, { useContext, useEffect, useReducer, useState } from 'react'
import { app } from '../../initFirebase'
import { getFirestore, collection, query, orderBy, where, onSnapshot, deleteDoc, doc, setDoc } from 'firebase/firestore'
import { Row, Col, Button } from 'react-bootstrap'
import TextareaAutosize from 'react-textarea-autosize'
import { ModalContext } from '../../context/ModalContext'

const reducer = (comments, action) => {
  switch(action.type){
    case 'call':
      return action.rows.map((row, index)=>row && {...row, index, isEllipsis:true, isEdit:false, text:row.content});
    case 'ellipsis':
      return comments.map(c=>c.id===action.comment.id ? { ...c, isEllipsis:!c.isEllipsis } : c);
    case 'edit':
      return comments.map(c=>c.id===action.comment.id ? { ...c, isEdit:true } : c);
    case 'change':
      return comments.map(c=>c.id===action.comment.id ? { ...c, content:action.value } : c);
    case 'cancel':
      return comments.map(c=>c.id===action.comment.id ? { ...c, isEdit:false, content:c.text } : c);
    default:
      return comments;
  }
}

const CommentList = ({ postId }) => {
  const [loading, setLoading] = useState(false);
  const { setConfirm } = useContext(ModalContext);
  const [comments, dispatch] = useReducer(reducer, []);
  const db = getFirestore(app);

  const callAPI = () => {
    setLoading(true);
    const q=query(collection(db,'comments'),where('postId','==',postId ),orderBy('date','desc'));
    onSnapshot(q, snapshot=>{
      const rows=[];
      snapshot.docs.forEach(row=>{
        rows.push({ id:row.id, ...row.data() });
      });
      //console.log(rows);
      dispatch({ type:'call', rows });
    });
    setLoading(false);
  }

  useEffect(()=>{
    callAPI();
  }, []);

  const onClickDelete = (comment)=> {
    setConfirm({
      show:true,
      message:`${comment.id}번 댓글을 삭제하실래요?`,
      action: async()=>{
        await deleteDoc(doc(db, 'comments', comment.id));
      }
    });
  }

  const onSave = async(comment)=> {
    await setDoc(doc(db, 'comments', comment.id), comment);
  }

  if(loading) return <h1 className='text-center my-5'>로딩중...</h1>
}
```

```

return (
  <div>
    { comments.map((comment,index)=>
      <div className='comment'>
        <Row className='mb-1'>
          <Col className='email mt-2'>{comment.email} | {comment.date}</Col>
          /*편집상태가 아니면서 내가 쓴 댓글인 경우만 보이기*/
          { !comment.isEdit && comment.email===sessionStorage.getItem('email') &&
            <Col className='text-end'>
              <Button onClick={()=>dispatch({ type:'edit', comment })}
                size='sm' variant='link'>수정</Button>
              <Button onClick={()=>onClickDelete(comment)}
                size='sm' variant='link'>삭제</Button>
            </Col>
          }
        </Row>
        /*편집상태가 아닌 경우에만 보이기*/
        { !comment.isEdit &&
          <div onClick={()=>dispatch({ type:'ellipsis', comment })}
            className={`content ${comment.isEllipsis && 'text-truncate'}}`>
              {index+1}. {comment.content}
            </div>
          }
        /*편집상태인 경우에만 보이기*/
        { comment.isEdit &&
          <React.Fragment>
            <TextareaAutosize onChange={(e)=>dispatch({ type:'change', comment, value:e.target.value })}
              value={comment.content} className='textarea' minRows={3}/>
            <div className='text-end'>
              <Button onClick={()=>onSave(comment)}
                disabled={ comment.text===comment.content } className='me-2' size='sm'>저장</Button>
              <Button onClick={()=>dispatch({ type:'cancel', comment })}
                size='sm' variant='secondary'>취소</Button>
            </div>
          </React.Fragment>
        }
      </div>
    )}
  </div>
)
}
export default CommentList

```
